4.2 USER-DEFINED FUNCTIONS IN PYTHON

What is a Function?

A **function** is a block of organized and reusable code designed to perform a specific task.
Functions help reduce repetition and make programs easier to manage.

Advantages of Functions

- ❖ Reuse code multiple times.
- ❖ Reduce code duplication.
- ❖ Improve program readability.
- ❖ Make debugging easier.
- ❖ Support modular programming.

Modularity

Modularity means dividing a large program into smaller and manageable parts called functions or modules.

Types of Functions in Python

There are two types of functions:

- ❖ Built-in Functions
- ❖ User-defined Functions

1. Built-in Functions (Library Functions)

Functions already provided by Python are called built-in functions.
These functions are automatically available.

Common Built-in Functions

Function	**Purpose**	Examples
print( )	Displays output	print("Hello")
input( )	Takes input	
len( )	Returns length	len("Python")
int( )	Converts into integer	int("5")
sum( )	Calculates total	sum([1,2,3])

2. User-defined Functions

A function created by the programmer using the def keyword is called a user-defined function.

Example

```
def greet( ):
   print("Hello")
```

Difference Between Built-in and User-defined Functions

Feature	Built-in Function	User-defined Function
Creator	Python Developers	Programmer
Availability	Available by default	Must be created
Loading	Loaded automatically	Loaded when defined
Example	print( )	add( )

Deepak Shrestha

Creating a User-defined Function

Syntax

```
def function_name(parameters):
    statements
    return value
```

Example Program

```
def add_numbers(x, y):
    total = x + y
    return total

print(add_numbers(5, 6))
```

Output

11

Important Rules of Functions

- Function definition starts with def
- Function header ends with colon :
- Indentation is compulsory
- Function name should be meaningful
- Parameters are optional
- Return statement is optional

Return Statement

A statement used to send a value back from a function.

Syntax

```
return value
```

Function Returning Value

Example

```
def area(length, width):
    return length * width

result = area(5, 8)

print(result)
```

Output

40

Parameters and Arguments

Parameter

A variable written inside the function definition is called a parameter.

Example

```
def area(radius):
```

radius is a parameter.

DeepakShrestha

Argument

The actual value passed during the function call is called an argument.

Example

area(5)
5 is an argument.

Difference Between Parameter and Argument

Parameter	Argument
Placeholder	Actual value
Used in definition	Used in function call
Variable	Value

Example

```
def add(x, y):
    return x + y

add(5, 3)
```

Item	Type
x, y	Parameters
5, 3	Arguments

Scope of Variables

Scope determines where a variable can be accessed.

Types of Scope

- Local Scope
- Global Scope

Local Scope

Variables declared inside a function and accessible only within that function.

Example

```
def test():
    num = 5
    print(num)

test( )
```

Global Scope

Variables declared outside functions and accessible throughout the program.

Example

```
num = 10

def show( ):
    print(num)

show( )
```

Deepak Shrestha

Passing Arguments

Python supports three types of arguments.

- ❖ Positional Arguments
- ❖ Default Arguments
- ❖ Keyword Arguments

1. Positional Arguments

Arguments are passed in the same order as parameters.

Example

```
def check(a, b, c):
    print(a, b, c)

check(2, 5, 7)
```

Output

2 5 7

2. Default Arguments

Parameters are assigned default values.

Example

```
def power(base, exp=2):
    return base ** exp

print(power(4))
print(power(2,3))
```

Output

16
8

Important Rule for Default Arguments

- Required parameters first
- Default parameters last

Legal Function

```
def interest(prin, time, rate=0.10):
```

Illegal Function

```
def interest(prin, time=2, rate):
```

- Error occurs because default arguments must come after required arguments.

3. Keyword Arguments

Arguments are passed using parameter names.

Order does not matter.

Example

```
def interest(prin, time, rate):
    pass

interest(rate=0.10, prin=2000, time=2)
```

Deepak Shrestha

Types of Functions Based on Return Value

- Void Function
- Non-Void Function

Void Function

A function that does not return any value.

Example

```
def greet( ):
   print("Hello")
```

Python automatically returns None.

Example

```
def greet():
   print("Hello")

a = greet( )

print(a)
```

Output

Hello
None

Non-Void Function

A function that returns a value.

Example

```
def add(x, y):
   return x + y

result = add(5,3)
```

Returning Multiple Values

Python can return multiple values using one return statement.

Example

```
def squared(x, y, z):
   return x*x, y*y, z*z

print(squared(2,5,7))
```

Output

(4, 25, 49)

Example with Variables

```
def squared(x, y, z):
   return x*x, y*y, z*z

a, b, c = squared(2,3,4)

print(a, b, c)
```

Output

4 9 16

Deepak Shrestha

Common Exam Mistakes □

- ❖ Forgetting colon : after function definition
- ❖ Wrong indentation
- ❖ Confusing parameter and argument
- ❖ Writing default arguments before required arguments
- ❖ Forgetting to call the function

Most Important Programs for SEE Exam

Program 1: Add Two Numbers

```
def add(a, b):
    return a + b

print(add(5, 3))
```

Program 2: Find Area of Rectangle

```
def area(length, width):
    return length * width

print(area(5, 8))
```

Program 3: Default Argument

```
def power(base, exp=2):
    return base ** exp

print(power(4))
```

Program 4: Keyword Argument

```
def student(name, age):
    print(name, age)

student(age=15, name="Ram")
```

Program 5: Local and Global Variable

```
x = 10

def show( ):
    y = 5
    print(x)
    print(y)

show( )
```

Deepak Shrestha

4.3 MODULE, PACKAGE AND LIBRARY IN PYTHON

Introduction

Python provides reusable code using modules, packages, and libraries. These help programmers write organized, reusable, and efficient programs.

Easy Memory Trick

Term	Meaning	Example
Module	Single Python file	math.py
Package	Folder containing modules	numpy.random
Library	Collection of packages and modules	NumPy, Pandas

Easy Analogy

Library = House
Package = Room
Module = Book
A house contains rooms, and rooms contain books.
Similarly:
Library → Packages → Modules

MODULE

Definition

A module is a single Python file (.py) containing reusable code such as functions, classes, variables, and statements.
Modules help reduce repetition and make programs organized.

Types of Modules

1. Built-in Module

A built-in module is a module already provided by Python that contains predefined functions, classes, and variables to perform specific tasks.
These modules are available automatically after Python installation.

Examples

- math
- random
- os
- sys
- datetime

Example Program

```
import math

print(math.sqrt(25))
```

Output

5.0

2. User-defined Module

A user-defined module is a module created by the programmer to store reusable code such as functions, classes, and variables in a separate Python file.

Deepak Shrestha

Example

```
# calculator.py

def add(a, b):
    return a + b
```

Using the Module

```
import calculator

print(calculator.add(5, 3))
```

Output

8

Advantages of Modules

1. Reuse code multiple times
2. Reduce code duplication
3. Make programs organized
4. Simplify testing and debugging
5. Improve readability

Methods of Importing Modules

1. Import Entire Module

```
import math
```

Example

```
print(math.factorial(5))
```

Output

120

2. Import Specific Function

```
from math import sqrt
```

Example

```
print(sqrt(49))
```

Output

7.0

3. Import Module Using Alias

An alias is an alternative short name given to a module during import using the as keyword.

Example

```
import math as m

print(m.sqrt(64))
```

Output

8.0

4. Import all functions from a module

Importing all functions from a module means importing every function, variable, and object from a module into the current program using the * symbol.

```
from module_name import *
```

Example:

Deepak Shrestha

```
from math import *

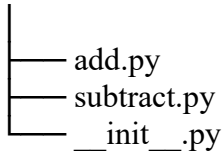
print(sqrt(25))
print(factorial(5))
```

Output
5.0
120

Important Point
After importing with *, functions can be used directly without writing the module name.
Example:
sqrt(25)
Instead of:
math.sqrt(25)

PACKAGE
Definition
A package is a collection of related Python modules organized inside a folder or directory.
Packages help organize large Python projects systematically.

Structure of a Package
calculator/



init.py File
__init__.py is a special file used to indicate that a directory should be treated as a Python package.

Advantages of Packages
1. Groups related modules together
2. Organizes large projects
3. Improves code management
4. Avoids naming conflicts
5. Supports hierarchical structure

Example of Package Import
```
from calculator.add import add
```

Important Points About Packages
- Contains one or more modules
- Usually includes __init__.py
- Can contain sub-packages
- Provides hierarchical organization

Deepak Shrestha

LIBRARY

Definition

A library is a collection of modules and packages that provides reusable functionality for specific tasks. Libraries help programmers perform complex tasks easily and quickly.

Types of Libraries

1. Standard Library

A standard library is a collection of built-in modules that are automatically available with Python installation.

Examples

- math
- random
- os
- datetime

---

2. External Library

An external library is a library that is not included with Python by default and must be installed separately using pip.

Examples

- NumPy
- Pandas
- Matplotlib
- TensorFlow
- OpenCV
- Django

Popular Python Libraries

Library	Main Use
NumPy	Numerical Computing
Pandas	Data Analysis
Matplotlib	Data Visualization
Requests	API Calls
TensorFlow	Machine Learning
OpenCV	Computer Vision
Django	Web Development

---

pip

Definition

pip is Python's package manager used to install, update, and remove external packages and libraries.

Common pip Commands

Install Package

pip install numpy

Uninstall Package

pip uninstall numpy

Show Installed Packages

pip list

Check pip Version

pip --version

Deepak Shrestha

Advantages of pip

1. Installs external libraries easily
2. Updates packages
3. Removes unwanted packages
4. Manages dependencies
5. Simplifies package management

Difference Between Module, Package and Library

Feature	Module	Package	Library
Meaning	Single Python file	Collection of modules	Collection of packages and modules
Size	Smallest	Medium	Largest
Form	.py file	Folder/Directory	Complete collection
Purpose	Reuse code	Organize modules	Provide complete functionality
Example	math.py	numpy.random	NumPy

Common Exam Mistakes

- Confusing module and package
- Writing package as a single file
- Forgetting.py extension in module definition
- Writing pip as a library instead of package manager

Final Summary

A module is a single Python file containing reusable code. A package is a collection of related modules stored inside a folder. A library is a collection of modules and packages that provides reusable functionality for developers.

Deepak Shrestha

4.4 GRAPHICS IN PYTHON USING TURTLE

Introduction to Turtle

Turtle is a Python module used to draw shapes, patterns, and pictures on the screen by moving a virtual pen called a turtle.
It is mainly used to teach programming concepts in a visual and interesting way.
It helps students learn programming through drawings and graphics.
It can draw shapes, lines, circles, patterns, and pictures easily.

Features of Turtle

Feature	Description
Easy Learning	Helps learn programming through drawings and graphics.
Fun Learning	Makes coding interesting by creating shapes and designs.
Creativity	Allows users to create different drawings and patterns.
Easy Error Detection	Mistakes can be identified easily by looking at the drawing.
Learn Shapes	Helps understand shapes, angles, and positions.
Better Thinking	Improves logical thinking by following drawing steps.

Basic Structure of a Turtle Program

```
import turtle

screen = turtle.Screen( )
pen = turtle.Turtle( )

# drawing commands

turtle.done( )
```

Explanation of Basic Turtle Statements

1. import turtle

Definition

import turtle is used to import the Turtle module so that Turtle graphics commands can be used in a Python program.

Simple Meaning

Python loads turtle graphics so drawing commands can work.

Without It

Turtle programs will not work.

2. import turtle as t

Definition

import turtle as t imports the Turtle module and gives it a short alias name t.

Simple Meaning

Instead of writing turtle every time, we can write only t.

Example

```
import turtle as t

pen = t.Turtle()
```

Deepak Shrestha

3. screen = turtle.Screen()

Definition

screen = turtle.Screen() creates a drawing window where the turtle can move and draw.

Simple Meaning

It opens a blank screen or canvas for drawing.

Used For

1. Changing background color
2. Setting window size
3. Changing window title
4. Detecting keyboard and mouse actions

---

4. pen = turtle.Turtle()

Definition

pen = turtle.Turtle() creates a turtle object and stores it in the variable pen.

Simple Meaning

It creates the drawing turtle that moves and draws on the screen.

---

5. turtle.done()

Definition

turtle.done() keeps the Turtle graphics window open until it is closed manually.

Simple Meaning

It prevents the drawing window from closing immediately after the program finishes.

---

6. turtle.mainloop()

Definition

turtle.mainloop() keeps the Turtle window open and waits for user actions.

Simple Meaning

It allows the program to keep running until the user closes the window.

Uses

1. Keeps the window open
2. Waits for keyboard or mouse events
3. Prevents the program from ending immediately

Important Turtle Commands

---

1. Turtle Motion Commands

Turtle motion commands are used to move and turn the turtle on the screen.

Command	Definition	Syntax	Example
forward() / fd()	Moves the turtle forward by a specified distance.	forward(distance)	pen.forward(100)
backward() / bk() / back()	Moves the turtle backward by a specified distance.	backward(distance)	pen.backward(50)
right() / rt()	Turns the turtle clockwise by a specified angle.	right(angle)	pen.right(90)
left() / lt()	Turns the turtle anticlockwise by a specified angle.	left(angle)	pen.left(90)

Deepak Shrestha

2. Circle Command

Command	Definition	Syntax	Example
circle()	Draws a circle or arc with a specified radius.	circle(radius)	pen.circle(50)

---

3. Color and Style Commands

Color and style commands are used to change the appearance of drawings.

Command	Definition	Syntax	Example
color()	Sets the line color of the turtle pen.	color(color_name)	pen.color("red")
fillcolor()	Sets the color used to fill a closed shape.	fillcolor(color_name)	pen.fillcolor("yellow")
pencolor()	Sets only the outline color of the turtle pen.	pencolor(color_name)	pen.pencolor("blue")
pensize()	Sets the thickness of the turtle pen.	pensize(size)	pen.pensize(5)
speed()	Controls the drawing speed of the turtle.	speed(value)	pen.speed(0)

Turtle Speed Values

Speed Value	Meaning
0	Fastest
1	Slowest
3	Slow
5	Medium
10	Fast

---

4. Pen Control Commands

Pen control commands control whether the turtle draws while moving.

Command	Definition	Syntax	Example
penup() / pu() / up()	Lifts the pen and stops drawing.	penup()	pen.penup()
pendown() / pd() / down()	Puts the pen down and starts drawing.	pendown()	pen.pendown()

Example

```
pen.penup()
pen.forward(100)
pen.pendown()
pen.forward(100)
```

Explanation

The first movement draws nothing because the pen is up.
The second movement draws a line because the pen is down.

---

5. Position Commands

Position commands control the turtle's location on the screen.

Command	Definition	Syntax	Example
goto()	Moves the turtle to a specific coordinate.	goto(x, y)	pen.goto(100, 50)
setpos() / setposition()	Moves the turtle to a specific coordinate.	setpos(x, y)	pen.setpos(50, 80)
home()	Returns the turtle to the starting position.	home()	pen.home()

Turtle Coordinate System

Direction	Coordinate Value
Right	Positive X (+X)
Left	Negative X (-X)
Up	Positive Y (+Y)
Down	Negative Y (-Y)

 The center of the Turtle screen is (0, 0).

Deepak Shrestha

6. Fill Commands

Fill commands are used to fill closed shapes with color.

Command	Definition	Syntax	Example
begin_fill()	Starts filling a shape with color.	begin_fill()	pen.begin_fill()
end_fill()	Ends filling and fills the shape.	end_fill()	pen.end_fill()

Example

```
pen.fillcolor("yellow")
pen.begin_fill()

for i in range(4):
    pen.forward(100)
    pen.right(90)

pen.end_fill()
```

7. Text Command

Command	Definition	Syntax	Example
write()	Displays text on the screen.	write(text)	pen.write("Hello Python")

Example

```
pen.write("Welcome to Turtle Graphics")
```

8. Screen and Turtle Control Commands

Command	Definition	Syntax	Example
clear()	Removes all drawings made by the turtle from the screen.	clear()	pen.clear()
reset()	Clears the screen and returns the turtle to its starting state.	reset()	pen.reset()
hideturtle()	Hides the turtle cursor.	hideturtle()	pen.hideturtle()
showturtle()	Displays the turtle cursor.	showturtle()	pen.showturtle()
bgcolor()	Changes the screen background color.	screen.bgcolor(color)	screen.bgcolor("lightblue")
title()	Sets the title of the Turtle window.	screen.title(title)	screen.title("My Drawing")

9. Shape Command

Definition

The shape() function is used to change the appearance of the turtle cursor.

Syntax

```
pen.shape("shape_name")
```

Example

```
pen.shape("turtle")
```

Common Turtle Shapes

Shape Name	Description
"arrow"	Default arrow cursor
"turtle"	Turtle-shaped cursor
"circle"	Circle-shaped cursor
"square"	Square-shaped cursor
"triangle"	Triangle-shaped cursor
"classic"	Classic turtle cursor

Deepak Shrestha

Common Exam Mistakes

1. Forgetting to write import turtle
2. Forgetting parentheses in commands
3. Writing forward instead of forward( )
4. Confusing penup() and pendown( )
5. Forgetting turtle.done( )
6. Using wrong indentation inside loops
7. Confusing angle and distance

Deepak Shrestha

4.5 ERROR HANDLING IN PYTHON

Error Handling

Error handling in Python is the process of detecting and handling errors during program execution using try, except, else, and finally blocks so that the program does not stop suddenly.

Errors and Exceptions

Errors

An error is a problem in a program that prevents the program from completing its task properly.
Errors may occur due to wrong syntax, incorrect logic, or invalid operations.

Debugging

Debugging is the process of finding and correcting errors or bugs in a program.

Types of Errors

There are mainly three types of errors in Python.

1. Syntax Errors
2. Runtime Errors / Exceptions
3. Logical Errors

1. Syntax Error

A syntax error is an error that occurs when the rules of Python syntax are violated while writing a program.
Syntax errors are detected before the program executes.

Causes of Syntax Error

- Missing colon :
- Missing brackets
- Wrong indentation
- Spelling mistakes in keywords

Example

```
if 5 > 2
   print("Hello")
```

Error

SyntaxError

2. Runtime Error / Exception

A runtime error is an error that occurs during the execution of a program after it has been successfully compiled.
An exception is a runtime error that interrupts the normal flow of a program.

Causes of Runtime Error

- Division by zero
- Invalid input
- Accessing invalid index
- File not found

Example

```
a = 10 / 0
```

Error

ZeroDivisionError

Deepak Shrestha

3. Logical Error

A logical error is an error in which the program runs successfully but produces incorrect output.
Logical errors are difficult to detect because no error message is shown.

Example

```
a = 5
b = 3

print(a - b)
```

Expected addition was intended, but subtraction was used.

Exception Handling

Exception handling is the process of handling errors in a program using try, except, else, and finally blocks to prevent the program from stopping unexpectedly.

Components of Exception Handling

1. try Block
2. except Block
3. else Block
4. finally Block

1. try Block

A try block is a block used to test a block of code for errors.
If an exception occurs, Python immediately transfers control to the except block.

Syntax

```
try:
   # risky code
```

Important Points

- Used for code that may generate errors
- Prevents sudden program crash
- Must be followed by except or finally block

2. except Block

The except block is a block used to handle errors or exceptions that occur in the try block.

Syntax

```
except:
   # handling code
```

Important Points

- Executes only when an error occurs
- Prevents abnormal termination of program
- Can handle specific exceptions

Example

```
try:
   a = 10 / 0
except ZeroDivisionError:
   print("Cannot divide by zero")
```

Deepak Shrestha

Output
Cannot divide by zero

3. else Block
The else block is a block that executes only when no exception occurs in the try block.
The else block is optional and must come after all except blocks.

Syntax
```
else:
   # code if no error occurs
```

Example
```
try:
   a = 10 / 2
except ZeroDivisionError:
   print("Error")
else:
   print(a)
```

Output
5.0

4. finally Block
The finally block is a block that always executes whether an exception occurs or not.
It is mainly used for cleanup tasks such as closing files, releasing resources, or ending database connections.

Syntax
```
finally:
   # cleanup code
```

Example
```
try:
   a = 10 / 0
except:
   print("Error")
finally:
   print("Program Ended")
```

Output
Error
Program Ended

Complete Example of Exception Handling
```
try:
   numerator = int(input("Enter numerator: "))
   denominator = int(input("Enter denominator: "))

   result = numerator / denominator

except ZeroDivisionError:
   print("Cannot divide by zero")
```

Deepak Shrestha

```
else:
    print("Result:", result)

finally:
    print("Program execution completed")
```

Output

```
Enter numerator: 10
Enter denominator: 0
Cannot divide by zero
Program execution completed
```

Advantages of Exception Handling

1. Prevents program crash
2. Improves program reliability
3. Makes debugging easier
4. Provides better user experience
5. Handles errors gracefully

Disadvantages of Exception Handling

1. Increases code length
2. Slightly slows program execution
3. Difficult for beginners
4. Improper handling may hide actual errors

Main Types of Exceptions in Python

1. ZeroDivisionError

ZeroDivisionError occurs when a number is divided by zero.

Example

```
a = 10 / 0
```

---

2. NameError

NameError occurs when a variable or function name is incorrect or undefined.

Example

```
print(name)
```

---

3. TypeError

TypeError occurs when incompatible data types are used together.

Example

```
a = "5" + 2
```

---

4. ValueError

ValueError occurs when the correct type of data is used but with an invalid value.

Example

```
num = int("abc")
```

---

5. IndexError

IndexError occurs when an invalid index is used in a list or sequence.

Example

```
list1 = [1, 2, 3]
```

DeepakShrestha

```
print(list1[5])
```

6. FileNotFoundError

FileNotFoundError occurs when a file being opened does not exist.

Example

```
open("data.txt")
```

---

7. Exception as e

Exception as e is used to capture the actual error object and display the error message.

Example

```
try:
   a = 10 / 0
except Exception as e:
   print(e)
```

Output

```
division by zero
```

Common Exam Mistakes

- Writingexpect instead of except
- Forgetting colon: after try and except
- Wrong indentation
- Confusing syntax error and logical error
- Forgetting that finally always executes

Deepak Shrestha